

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

2026.04.17

JavaScript 기초와 DOM 조작

출처	D:\WStudy\WHTMLW260417_JS.html
변수	var, let, const의 차이와 const 재할당 불가 확인
DOM 접근	getElementById, querySelector로 HTML 요소 선택
내용 변경	innerText, innerHTML로 텍스트와 내부 HTML 수정
스타일 변경	style.color, style.fontSize, backgroundColor처럼 JS에서 CSS 조작
클래스 변경	classList.add/remove/toggle로 CSS 클래스 제어
조건 표현	삼항연산자로 조건에 따라 문구를 바꿈

수업 코드 원본 (D:\WStudy)

핵심	아래는 D:\WStudy에 저장된 실제 수업 코드 원본이다. 설명으로 대체하지 않고, 먼저 원본을 보준한 뒤 바로 아래에 짧게 의미를 붙인다.
----	---

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

파일: D:\Study\WHTML\W260417_JS.html

```
<!DOCTYPE html>
<html lang="en">

<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<style>
.red {
color: red !important;
}
</style>
</head>

<body>

<h1 id="title">어쩌고 타이틀</h1>

<button onclick="toggleTitle()">확인</button>



<script>
(function (){ //즉시실행함수
var a = 1; // 옛날버전, 전역 scope, if, for 내부에서 생성 시 외부에도 영향을 줌
let b = 2; // 요즘 버전 변수 만드는법
const c = 3; // 값을 변경할 수 없는 상수
// c = c + 1; // 에러

// DOM 접근 = Document Object Model = HTML을 JS객체로 다룬다 = js로 html을 조작한다
var title1 = document.getElementById("title"); // id로 찾기
var title2 = document.querySelector("#title"); // CSS Selector처럼사용 id는 #title class는 .title

// 내용 조작부
title1.innerText = "텍스트 변경"
title1.innerHTML = "내부 <mark>HTML</mark> 변경"

// css 스타일 변경
title1.style.color = "blue"; // style.css속성명(카멜케이스)
title1.style.fontSize = "24px";
title1.style.backgroundColor = '#dfdfff';

//class변경
```

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```

title1.classList.add('red'); // 클래스 붙이기
title1.classList.remove('red'); // 클래스 지우기
title2.classList.toggle('red'); // on이면 off, off면 on 해주는기능

// ~~~변경
var img = document.getElementById('img');
img.src = "/images/bg.jpg"
img.width = '200';

//isNaN(변수), 변수값이 상수인지 아닌지 판별
console.log(isNaN(a)); // TRUE / FALSE 값 리턴

//삼항연산자
// (조건) ? 참일때값 : 거짓일때값
var num = prompt('차 번호를 입력하세요. ');
var today = new Date();
var toDate = today.getDate();
console.log(toDate);

title1.innerText = num % 2 == toDate % 2 ? '운행가능합니다.' : '오늘은 쉬세요!';

})();

function toggleTitle() {
var title1 = document.getElementById('title');
title.classList.toggle('red');
}

</script>
</body>

</html>

```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 DOM 선택 후 innerText/innerHTML/style/classList로 화면을 바꾸는 코드다. / 폼 입력값이 서버 라우트로 전달되는 구조를 확인한다.

원본 위치	D:\Study\HTML\260417_JS.html
설명	이 날짜 수업에서 작성/참고한 '260417_JS.html' 코드 원본

수업 코드 원본 끝

subject	title	check
김태형교수님	김태형교수님 수업정리	☐ ☐ ☐ ☐ ☐
		date

2026.05.07

JS 계산기, fetch, 로컬 AI 요청

출처	D:\WStudy\WHTMLW260507.html
입력	date, number, text input 값을 읽어 계산
BMI 계산	체중 / 키(m)^2 공식을 사용하고 결과 범위에 따라 문구/색상을 바꿈
fetch	HTTP 요청을 보내고 then(r => r.json())으로 응답 처리
JSON	headers Content-Type과 JSON.stringify로 API 요청 본문 구성
화면 출력	textContent/innerHTML로 결과 영역 업데이트

수업 코드 원본 (D:\WStudy)

핵심

아래는 D:\WStudy에 저장된 실제 수업 코드 원본이다. 설명으로 대체하지 않고, 먼저 원본을 보존한 뒤 바로 아래에 짧게 의미를 붙인다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

파일: D:\WStudy\WHTML\W260507.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>간단js 계산기</title>
<style>
main{
max-width: 620px;
margin: auto;
}
fieldset
{
margin-bottom: 40px;
}
</style>
</head>
<body>
<main>
<fieldset>
<legend>나이 계산</legend>
<input type="date" id ="date" value = "1994-08-18">
<button onclick = "calcAge()">나이 계산</button>
</fieldset>
<fieldset>
<legend>BMI 계산</legend>

키(cm):
<input type="number", id = "height", min = "1", max = 200, value = "183">

체중(kg):
<input type="number", id = "weight", min = "1", max = 200, value = "105">

<button onclick= "calcBMI()" >BMI확인</button>
<!-- bmi = 체중 / (키*키(m))-->
<!-- 18.5미만: 저체중, 23미만 : 정상, 25미만: 과체중, 25이상 비만-->
<!-- result color = 저체중: 파랑, 정상: 초록, 과체중: 주황, 비만: 빨강-->
<div>
결과: 당신의 BMI수치는 <span id="bmi">--</span>이며 <span id ="result">--</span>입니다.
</div>
</fieldset>
<fieldset>
<legend>유튜브 정보 가져오기</legend>

```

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```

<input type="text" id = "youtubeUrl" placeholder = "동영상 주소를 입력해 주세요" value
="https://www.youtube.com/watch?v=GSWavfzSVnE">
<button onclick = "getYoutube()">정보 가져오기</button>
<div id="youtubeResult"></div>
</fieldset>
<fieldset>
<legend>로컬 AI에게 질문하기</legend>
<input type="text" id = "aiInput" placeholder = "질문을 입력해주세요" value = "안녕">
<button onclick = "requestLocalAI()">질문하기</button>
<div id = "aiResult"></div>
</fieldset>
</main>

<script>
function requestLocalAI(){
const input = document.getElementById('aiInput').value;
if(!input)
{
alert('질문을 입력해주세요');
return;
}
const headers = new Headers();
headers.append("Content-Type", "application/json");

fetch('https://192.168.10.147:1234/api/v1/chat', {
method: "POST",
headers: headers,
body: JSON.stringify({
"model": "qwen2.5-vl-7b-instruct",
"input": input,
"store": false
})
}).then(r => r.json()).then(r => {
console.log(r); //실제 응답 결과
document.getElementById('aiResult').textContent = r.output[0].content;
});

document.getElementById('aiInput').value = ""
}
function getYoutube()
{
const youtubeUrl = document.getElementById('youtubeUrl').value;
if(!youtubeUrl || !youtubeUrl.startsWith('https://www.youtube.com'))
{
alert('올바른 유튜브 동영상 주소를 입력해주세요');
return;
}
}

```

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```

}

const apiUrl = "http://www.youtube.com/oembed?url=" + youtubeUrl;
// apiUrl에 저장된 주소로 http요청을 전송, 응답을 받아와서 html을 생성해서 #youtubeResult에 세팅한다.
fetch(apiUrl).then(r => r.json()).then(r => {
  document.getElementById("youtubeResult").innerHTML = `<div>${r.title}</div><div><img width = "200" src =
"${r.thumbnail_url}"</div>`;
});

}

function calcAge()
{
  let age = 0;
  //현재년도
  const date = Number((new Date()).getFullYear());
  const birth = Number(document.getElementById('date').value.slice(0, 4));
  age = date - birth;

  alert('당신의 나이는' + age + '살 입니다');
}

function calcBMI()
{
  let cm = document.getElementById('height').value;
  let kg = document.getElementById('weight').value;
  let bmi = kg / ((cm/100)*(cm/100));
  let result= document.getElementById('result')
  if (!cm || !kg)
  {
    alert('키 또는 무게를 입력 해주세요');
    return;
  }

  document.getElementById('bmi').innerText = bmi.toFixed(2);
  if(bmi < 18.5)
  {
    result.innerText = "저체중"
    result.style.color = "blue"
  }
  else if( bmi < 23)
  {
    result.innerText = "정상"
    result.style.color = "green"
  }
  else if(bmi < 25)
  {
    result.innerText = "과체중"
  }
}

```

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```

result.style.color = "orange"
}
else if(bmi >= 25)
{
result.innerText = "비만"
result.style.color = "red"
}
speak('당신의 BMI판정 결과는 ' + result + ' 입니다.');
```

```

}
```

```

function speak(text)
{
const speaking = new SpeechSynthesisUtterance(text);

speaking.lang = "ko_KR"
speaking.pitch = 1.0;
speaking.voice = voice;

speechSynthesis.speak(speaking);
}
</script>
</body>
</html>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 DOM 선택 후 innerText/innerHTML/style/classList로 화면을 바꾸는 코드다. / fetch로 HTTP 요청을 보내고 응답을 JSON 또는 텍스트로 처리한다. / 폼 입력값이 서버 라우트로 전달되는 구조를 확인한다.

원본 위치	D:\Study\HTML\W260507.html
설명	이 날짜 수업에서 작성/참고한 '260507.html' 코드 원본

수업 코드 원본 끝

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

2026.05.14

Node.js Express와 EJS

출처	D:\WStudy\WHTMLW260514\server.js, D:\WStudy\WHTMLW260514\views\index.ejs
서버	express()로 app 생성 후 app.listen(80)으로 HTTP 서버 실행
템플릿	app.set('view engine', 'ejs')로 EJS 사용
GET 라우트	app.get('/')에서 res.render('index', {chatData})로 화면 렌더링
POST 라우트	app.post('/chat')에서 req.body.message를 받아 배열에 저장
폼 처리	express.urlencoded({ extended: true })로 form 데이터를 읽음

수업 코드 원본 (D:\WStudy)

핵심	아래는 D:\WStudy에 저장된 실제 수업 코드 원본이다. 설명으로 대체하지 않고, 먼저 원본을 보준한 뒤 바로 아래에 짧게 의미를 붙인다.
----	---

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

파일: D:\Study\WHTML\W260514\server.js

```
const express = require('express');
const app = express();

app.set('view engine', 'ejs');
app.use(express.urlencoded({ extended: true }));

const chatData = [];

app.get('/', function(req,res){

  //메인화면 보여주기
  res.render('index', {chatData});
});

app.post('/chat', function(req,res){
  //채팅 메시지 저장처리
  const now = new Date();

  console.log(req.body.message);
  chatData.push(req.body.message + " (" + now.getHours() + ":" + now.getMinutes() + ")");

  res.redirect('/');
});

app.listen(80);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

Express 라우트: GET /, POST /chat / Express 서버 코드라서 미들웨어, 라우트, render/redirect 흐름을 본다.

원본 위치	D:\Study\WHTML\W260514\server.js
설명	이 날짜 수업에서 작성/참고한 `server.js` 코드 원본

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

파일: D:\Study\WHTML\260514\wviews\windex.ejs

```

<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>개쩌는 모두의 채팅</title>

<style>
.chat_list{
width: 400px;
height: 600px;

border: 1px solid gray;
padding: 10px;
overflow-y: auto;
border-radius: 5px;

display: flex;
flex-direction: column;
justify-content: flex-end;
}
.chat_list div{
margin-bottom: 10px;
background-color: #f1f1f1;
padding: 5px;
border-radius: 5px;
}
form{
width: 421px;
display: flex;
}
input{
flex: 1;
padding: 10px;
}
</style>
</head>
<body class = "chat_box">
<div class = "chat_list">
<!-- for -->
<% for( let i = 0; i < chatData.length; i++) {%>
<div><%=chatData[i]%></div>
<% } %>

```

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```

</div>
<form action="/chat" method = "post" onsubmit = "return chatInput()">
<input id = "message" type="text" name="message" placeholder="아무말이나 써부러보세요!">
<button type = "submit">전송</button>
</form>
<script>
function chatInput(){
const input = document.getElementById('message').value;

if(!input)
{
alert("채팅을 입력해주세요");
return false;
}
return true;
}
</script>
</body>
</html>

```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

DOM 선택 후 innerText/innerHTML/style/classList로 화면을 바꾸는 코드다. / 폼 입력값이 서버 라우트로 전달되는 구조를 확인한다.

원본 위치	D:\Study\HTML\260514\views\index.ejs
설명	이 날짜 수업에서 작성/참고한 `index.ejs` 코드 원본

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

파일: D:\Study\WHTML\Windex.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
</head>
<body>
<h1>개쩌는 메인화면</h1>
<form action="/welcome">
<input type="text" name ="name" required>
<button type="submit">확인</button>
</form>
</body>
</html>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 폼 입력값이 서버 라우트로 전달되는 구조를 확인한다.

원본 위치	D:\Study\WHTML\Windex.html
설명	이 날짜 수업에서 작성/참고한 `index.html` 코드 원본

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

파일: D:\Study\WHTML\index.js

```
const express = require('express')
const cheerio = require('cheerio')

const server = express()
const port = 80

console.log('express 서버 생성')

server.set('view engine', 'ejs')

// 메인 페이지
server.get('/', function(req, res) {

  console.log('/ 요청 옴!!')

  res.sendFile(__dirname + '/index.html')
})

// welcome 페이지
server.get('/welcome', function(req, res){

  console.log('웰컴 요청들어옴!!')

  const userName = req.query.name ?? '익명'

  console.log(req.query)

  res.render('welcome', { userName })
})

var cache = null;

// 메뉴 페이지
server.get('/menu', function(req, res){

  if (cache !== null)
  {
    console.log('캐시에서 처리함');
    res.render('menu', {menuList});
    return;
  }
}
```

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```

fetch('https://kopo.ac.kr/jinju/content.do?menu=6767')
.then(r => r.text())
.then(r => {

const menuList = parseMealTable(r)

console.log(menuList)

res.render('menu', { menuList })
})
.catch(err => {

console.log(err)

res.send('급식 데이터를 불러오는데 실패했습니다.')
})
})

// 뉴스 페이지
server.get('/news', (req, res) => {

res.send('This is news page')
})

// 서버 시작
server.listen(port, () => {

console.log('Example app listening on port ${port}')
})

console.log('서버시작')

// 급식 파싱 함수
function parseMealTable(html) {

const $ = cheerio.load(html)

const result = []

// 모든 tr 검사
$('tr').each((i, tr) => {

const tds = $(tr).find('td')

// td 4개 미만이면 스킵

```

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```

if (tds.length < 4) return

const firstTdHtml = $(tds[0]).html()

if (!firstTdHtml) return

// 날짜 추출
const dateMatch = firstTdHtml.match(/getDay2W('([Wd-]+)')/)

const date = dateMatch ? dateMatch[1] : null

// 날짜 없으면 스킵
if (!date) return

// 요일 추출
const dayMatch = $(tds[0])
.text()
.match(/(월요일|화요일|수요일|목요일|금요일|토요일|일요일)/)

const day = dayMatch ? dayMatch[1] : ''

// 메뉴 문자열 -> 배열 변환
const parseMenu = (td) => {

const text = $(td)
.text()
.replace(/\s+/g, ' ')
.trim()

if (!text) return []

return text
.split(',')
.map(v => v.trim())
.filter(Boolean)
}

result.push({

date,
day,

breakfast: parseMenu(tds[1]),
lunch: parseMenu(tds[2]),
dinner: parseMenu(tds[3]),

```


subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```

    })
  })

  return result
}

```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 fetch로 HTTP 요청을 보내고 응답을 JSON 또는 텍스트로 처리한다. / Express 서버 코드라서 미들웨어, 라우트, render/redirect 흐름을 본다.

원본 위치	D:\Study\HTML\index.js
설명	이 날짜 수업에서 작성/참고한 `index.js` 코드 원본

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

파일: D:\Study\WHTML\views\menu.ejs

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>급식 메뉴</title>
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@picocss/pico@2/css/pico.min.css">
</head>
<body>
<main class = "container">
<h1>이번 주 급식 메뉴</h1>

<%
/* 이 내부는 그냥 JavaScript 코드를 사용 할 수 있다*/
const today = (new Date()).getDay();
%>
<% for(let i = 0; i<menuList.length; i++) { %>
<article>
<details <%= i + 1 == today ? 'open' : '' %>>
<summary><%= menuList[i].day %></summary>
<ul>
<li>아침: <%= menuList[i].breakfast.join(' ') %></li>
<li>점심: <%= menuList[i].lunch.join(' ') %></li>
<li>저녁: <%= menuList[i].dinner.join(' ') %></li>
</ul>
</details>
</article>
<%} %>
</main>
</body>
</html>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 폼 입력값이 서버 라우트로 전달되는 구조를 확인한다.

원본 위치	D:\Study\WHTML\views\menu.ejs
설명	이 날짜 수업에서 작성/참고한 `menu.ejs` 코드 원본

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

파일: D:\Study\WHTML\views\welcome.ejs

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
</head>
<body>
<h1><%=userName%>님 환영합니다.</h1>
</body>
</html>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 폼 입력값이 서버 라우트로 전달되는 구조를 확인한다.

원본 위치	D:\Study\WHTML\views\welcome.ejs
설명	이 날짜 수업에서 작성/참고한 `welcome.ejs` 코드 원본

수업 코드 원본 끝

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

2026.05.15

TCP 소켓 통신

출처	D:\WStudy\WHTMLW260515Wtcp_server.py, D:\WStudy\WHTMLW260515Wtcp_client.py
socket	TCP 통신을 위한 소켓 객체 생성
서버 흐름	socket -> bind -> listen -> accept -> recv/send -> close
클라이언트 흐름	socket -> connect -> send/recv -> close
인코딩	문자열 전송 전 encode, 수신 후 decode가 필요
종료 조건	message == 'exit'이면 반복문을 빠져나와 연결 종료

수업 코드 원본 (D:\WStudy)

핵심	아래는 D:\WStudy에 저장된 실제 수업 코드 원본이다. 설명으로 대체하지 않고, 먼저 원본을 보준한 뒤 바로 아래에 짧게 의미를 붙인다.
----	---

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

파일: D:\Study\WHTML\W260515\tcp_client.py

```
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 접속 요청
client_socket.connect(('127.0.0.1', 8282))

while True:
    message = input('메시지 입력: ')
    # 접속 되면 바로 메시지 전송
    client_socket.send(message.encode()) # str -> byte로 변환해서 보내야함
    print('수신: ' + client_socket.recv(1024).decode())
    if message == 'exit':
        break

# 연결 종료

client_socket.close()
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 TCP 소켓 코드라서 socket -> bind/connect -> send/recv -> close 순서로 읽는다.

원본 위치	D:\Study\WHTML\W260515\tcp_client.py
설명	이 날짜 수업에서 작성/참고한 `tcp_client.py` 코드 원본

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

파일: D:\Study\HTML\260515\tcp_server.py

```
import socket

# ip주소 port번호

# 소켓 생성 (갤럭시 구입)
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 포트 바인딩 (전화번호 개통)
server_socket.bind(('127.0.0.1', 8282))

# 연결 대기 (전화 기다림)
server_socket.listen()
print('서버 소켓 대기')

# 실제로는 연결될때까지 여기서 멈춰있음ㅋ
# 전화가 옴!! 연결 요청이 오면 연결 수락

client_socket, client_ip = server_socket.accept()
print(f'연결되었습니다. {client_ip}')

## 데이터 수신 (전화 청취)
while True:
    message = client_socket.recv(1024).decode() # byte -> str
    print('수신: ' + message)
    client_socket.send('OK!'.encode())
    if message == 'exit':
        break

# 전화 끊기
client_socket.close()

# 전화기 가입 해지
server_socket.close()
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 TCP 소켓 코드라서 socket -> bind/connect -> send/recv -> close 순서로 읽는다.

원본 위치	D:\Study\HTML\260515\tcp_server.py
설명	이 날짜 수업에서 작성/참고한 `tcp_server.py` 코드 원본

수업 코드 원본 끝

subject	title	check
김태형교수님	김태형교수님 수업정리	☐ ☐ ☐ ☐ ☐
		date

2026.05.28

라즈베리파이(미니 PC ARM) => 리눅스 설치(dietpi, 데비안 리눅스 기반)

리눅스 명령어

ls	파일/폴더 목록 확인. ls -alh: a는 숨김파일 표시, l은 권한 및 용량까지 상세 리스트업, h는 사람이 읽기 쉬운 용량 단위 표시
mkdir	디렉토리(폴더) 생성. 예: mkdir xxxxx
cd 경로	해당 경로로 이동. . 현재폴더, .. 상위폴더, ~/ 사용자 홈 디렉토리
rm	파일 삭제. -r 옵션은 폴더 삭제, -f 옵션은 강제 삭제. 예: rm -rf xxxx
cp	파일 복사. 예: cp a.txt b.txt, 폴더 복사 cp -r a b
mv	파일 이동 또는 이름 변경. 예: mv a.txt b.txt 는 이름 변경, mv aaa ./ 는 aaa 폴더/파일을 상위로 이동
pwd	현재 폴더의 절대경로(root부터 시작하는 경로 전체) 확인
passwd	비밀번호 변경
htop	CPU, 메모리, 프로세스 확인. 비슷한 명령어: ps -elf
kill -9 xxx	xxx는 PID이며 프로세스를 강제 종료
cat	파일 내용을 화면에 출력
head, tail	파일 앞/뒤 일부 출력. 예: head -n 30 aaa.txt 는 상위 30줄 출력, tail -n 20 xxx.txt 는 맨 끝 20줄 출력
vi, nano	파일 편집
touch xxx.txt	파일 생성
grep 검색어 파일명	특정 파일에서 검색어를 찾음
grep -B 2 단어 파일명	검색된 단어 앞 2줄까지 같이 출력. -A 옵션은 after로 검색 결과 뒤쪽 줄을 같이 출력
파이프라인/리다이 렉션	ls > list.txt 는 출력 결과를 파일에 저장, ls >> list.txt 는 출력 결과를 파일에 추가
명령어1 명령어2	명령어1의 출력 결과를 명령어2의 입력으로 전달
예시	ls -al grep test
df -h	디스크 사용량 확인

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

네트워크/서버 확인 명령어

ip addr	IP 주소 확인
ping ip주소	해당 IP와 연결되는지 확인. 방화벽 문제 확인에도 사용
curl https	//www.naver.com: HTTP 요청을 보내고 응답 출력
curl -i 주소	응답 헤더까지 함께 출력
wget	파일 다운로드. 예: wget https://파일주소 -O 저장할파일명
ss -tunlp	서비스 중인 포트 목록 확인
lsof -i	8080: 8080 포트를 점유 중인 프로세스 찾기
nslookup	DNS 확인. 없으면 sudo apt install dnsutils 로 설치
chmod	파일/폴더 권한 변경

압축 명령어

zip xxx.zip 파일명	파일 압축
zip -r xxx.zip 폴더명	폴더 압축
unzip	zip 압축 해제

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

SW 설치와 서버 구축

samba	파일서버. 윈도우 파일 공유에 사용
nginx	웹서버(http 서버). 설정하는 법 학습
도메인 연결	dongsbe.kro.kr 사용
리버스 프록시	nginx를 앞단에 두고 내부 서버로 요청 전달
gitea	GitHub 대신 사용할 수 있는 저장소 서버 구축
node 서버	Node로 서버를 만들고 도메인 세팅 후 gitea 소스코드 배포까지 간단 버전으로 진행
현재 진행상황	라즈베리파이에 서버를 열어서 실습 중
사용 도메인	dongsbe.kro.kr, git.dongsbe.kro.kr

PuTTY 접속과 서버 패키지 설치 진행

PuTTY	윈도우에서 라즈베리파이 리눅스 서버에 SSH로 접속할 때 사용하는 터미널 프로그램
접속 대상	라즈베리파이 서버
진행 내용	PuTTY로 접속한 뒤 Gitea, nginx 등 서버 운영에 필요한 프로그램을 다운로드/설치하는 중
nginx	웹서버 및 리버스 프록시 역할
Gitea	GitHub 대신 직접 운영할 수 있는 Git 저장소 서버
흐름	SSH 접속 -> 패키지 다운로드/설치 -> 서비스 실행 확인 -> 도메인/리버스 프록시 연결
확인할 명령어	ip addr, ping, curl -i, ss -tunlp, lsof -i:포트번호, systemctl status 서비스명

라즈베리파이 Node 서버 배포 실행

- 라즈베리파이에 Git 저장소를 git clone으로 내려받음
- 내려받은 프로젝트 폴더에서 Node 서버 파일 index.js를 실행

실행 명령어	forever start index.js
forever	Node 프로세스를 백그라운드에서 계속 실행시키는 도구
의미	터미널을 닫아도 Node 서버가 계속 동작하도록 실행
배포 흐름	git clone -> 프로젝트 폴더 이동 -> 의존성 설치 필요 시 npm install -> forever start index.js
확인할 명령어	forever list, ss -tunlp, curl -i 도메인, systemctl status nginx

community.dongsbe.kro.kr 리버스 프록시 설정

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

nginx 설정 경로	/etc/nginx/sites-enable 에 community 설정 파일 생성
참고	일반적인 nginx 표준 경로 이름은 /etc/nginx/sites-enabled
목표 도메인	community.dongsbe.kro.kr
내부 서비스 포트	3030
구성 방식	nginx 리버스 프록시로 community.dongsbe.kro.kr 요청을 내부 3030 포트 서비스로 전달
의미	외부 사용자는 도메인으로 접속하고, 실제 Node/내부 서버는 3030 포트에서 동작
확인 흐름	nginx 설정 파일 작성 -> nginx 설정 검사 -> nginx 재시작/리로드 -> 도메인 접속 확인
점검 명령어	sudo nginx -t, sudo systemctl reload nginx, ss -tunlp, curl -i http://community.dongsbe.kro.kr

Git 배포 스크립트 커밋/푸시

- 실행한 명령어:
- git status
- git update-index --chmod=+x .Wdeploy.sh
- git add *
- git commit -m "배포 스크립트 추가"
- git push

deploy.sh	서버 배포 작업을 자동화하기 위한 쉘 스크립트로 보임
-----------	-------------------------------

- git update-index --chmod=+x .Wdeploy.sh: deploy.sh 파일에 실행 권한을 Git 인덱스에 기록

의미	서버를 바로 재실행하는 명령이 아니라, deploy.sh가 실행 가능한 파일이라는 권한 정보를 Git에 반영해서 커밋/푸시하는 과정
git add *	변경 파일을 스테이징
git commit -m "배포 스크립트 추가"	배포 스크립트 추가 내용을 커밋
git push	원격 저장소로 커밋 업로드
서버 자동 재실행과의 관계	deploy.sh 안에 git pull, npm install, forever restart 같은 명령이 들어있고 서버에서 실행되면 재배포/재시작 자동화가 가능함

코드/명령어 박스 예시 정리

```
git clone 저장소주소
```

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```
cd 프로젝트폴더
npm install
forever start index.js
forever list
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

git clone	라즈베리파이에 소스코드를 내려받음
npm install	필요한 Node 패키지 설치
forever start index.js	Node 서버를 백그라운드에서 계속 실행
forever list	실행 중인 forever 프로세스 확인

```
server {
  listen 80;
  server_name community.dongsbe.kro.kr;

  location / {
    proxy_pass http://127.0.0.1:3030;
  }
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

- community.dongsbe.kro.kr 요청을 nginx가 받아 내부 3030 포트의 Node/Express 서버로 전달
- 이 구조가 리버스 프록시

```
git status
git update-index --chmod=+x ./deploy.sh
git add *
git commit -m "배포 스크립트 추가"
git push
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

- git update-index --chmod=+x는 서버 재실행 명령이 아니라 deploy.sh 실행 권한을 Git에 기록하는 명령
- 실제 재시작은 deploy.sh 안의 forever restart, systemctl reload 같은 명령으로 처리

실제 deploy.sh 내용

```
#!/bin/bash

cd ~/Project/my-community
git pull
forever restart index.js
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

#!/bin/bash	이 파일을 bash 셸로 실행하겠다는 선언
cd ~/Project/my-community	서버에 clone된 my-community 프로젝트 폴더로 이동
git pull	원격 저장소의 최신 커밋을 서버로 가져옴
forever restart index.js	forever로 실행 중인 index.js Node 서버를 재시작
의미	deploy.sh를 실행하면 최신 코드 반영과 Node 서버 재시작이 한 번에 진행됨
정리	git update-index --chmod=+x는 실행 권한을 Git에 기록하는 단계이고, 실제 자동 배포/재시작은 이 deploy.sh 내용이 담당함

수업 중 작성/사용 코드 원본 보존

핵심 아래는 설명으로 풀어쓴 내용이 아니라, 수업 중 실제로 사용한 명령어/설정/스크립트 원본을 보존한 부분이다.

Git 커밋/푸시 명령어 원본

```
git status
git update-index --chmod=+x .Wdeploy.sh
git add *
git commit -m "배포 스크립트 추가"
git push
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

git status	현재 변경사항 확인
-------------------	------------

- git update-index --chmod=+x .Wdeploy.sh: deploy.sh 실행 권한을 Git 인덱스에 기록

git add *	변경 파일 전체 스테이징
git commit -m "배포 스크립트 추가"	커밋 생성
git push	원격 저장소로 업로드

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

deploy.sh 원본

```
#!/bin/bash

cd ~/Project/my-community
git pull
forever restart index.js
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

cd ~/Project/my-community	서버에 clone된 프로젝트 폴더로 이동
git pull	원격 저장소의 최신 코드 가져오기
forever restart index.js	forever로 실행 중인 Node 서버 재시작

라즈베리파이 Node 서버 실행 원본

```
git clone 저장소주소
cd 프로젝트폴더
npm install
forever start index.js
forever list
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

git clone	라즈베리파이에 저장소 내려받기
npm install	Node 의존성 설치
forever start index.js	서버 시작
forever list	forever 프로세스 목록 확인

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

nginx community 리버스 프록시 작업 원본

```
cd /etc/nginx/sites-enable
sudo nano community
sudo nginx -t
sudo systemctl reload nginx
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

cd /etc/nginx/sites-enable	수업에서 사용한 nginx 사이트 설정 경로
sudo nano community	community 설정 파일 작성
sudo nginx -t	nginx 문법 검사
sudo systemctl reload nginx	nginx 설정 리로드

nginx 리버스 프록시 설정 예시 원본

```
server {
    listen 80;
    server_name community.dongsbe.kro.kr;

    location / {
        proxy_pass http://127.0.0.1:3030;
    }
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

- community.dongsbe.kro.kr 요청을 내부 3030 포트 Node/Express 서버로 전달
- `proxy\_pass`가 리버스 프록시의 핵심

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

도메인/서버 확인 명령어 원본

```
curl -I http://git.dongsbe.kro.kr
curl -I https://git.dongsbe.kro.kr
curl -I http://community.dongsbe.kro.kr
ss -tunlp
lsof -i:3030
systemctl status nginx
systemctl status gitea
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

git.dongsbe.kro.kr	Gitea 서버 확인
community.dongsbe.kro.kr	Node/Express community 서버 확인
ss/lsof	포트 점유와 서비스 상태 확인
systemctl status	서비스 실행 상태 확인

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

리눅스 명령어 원본 묶음

```
ls
ls -alh
mkdir xxxxx
cd 경로
rm -rf xxxx
cp a.txt b.txt
cp -r a b
mv a.txt b.txt
mv aaa ../
pwd
passwd
htop
ps -elf
kill -9 PID
cat 파일명
head -n 30 aaa.txt
tail -n 20 xxx.txt
vi 파일명
nano 파일명
touch xxx.txt
grep 검색어 파일명
grep -B 2 단어 파일명
grep -A 2 단어 파일명
df -h
ip addr
ping ip주소
curl https://www.naver.com
curl -i 주소
wget https://파일주소 -O 저장할파일명
nslookup 도메인
chmod +x 파일명
zip xxx.zip 파일명
zip -r xxx.zip 폴더명
unzip 파일명.zip
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

정리

앞으로 수업정리에서는 네가 작성한 코드/명령어 원문을 먼저 보존하고, 그 아래에 설명을 붙이는 방식으로 처리한다.

MariaDB 외부 접속 설정

라즈베리파이에서 MariaDB 설정 파일을 열어 `bind-address`를 `0.0.0.0`으로 변경했다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```
cd /etc/mysql
ls
cd mariadb.conf.d/
ls
sudo nano 50-server.cnf
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 ``/etc/mysql/mariadb.conf.d/50-server.cnf``는 MariaDB 서버 설정 파일이다. 여기서 ``bind-address`` 값을 바꾸면 MariaDB가 어느 IP에서 접속을 받을지 정할 수 있다.

```
bind-address = 0.0.0.0
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 ``0.0.0.0``은 모든 네트워크 인터페이스에서 접속을 받겠다는 뜻이다. 기본값이 ``127.0.0.1``이면 라즈베리파이 자기 자신에서만 접속 가능하고, 외부 PC에서는 접속이 막힌다.

확인 설정을 바꾼 뒤에는 MariaDB 재시작과 포트 확인이 필요하다.

```
sudo systemctl restart mariadb
sudo systemctl status mariadb
ss -tunlp | grep 3306
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주의 ``bind-address = 0.0.0.0``만으로 접속이 전부 끝나는 것은 아니다. 외부 접속용 DB 계정 권한, 비밀번호, 방화벽/공유기 포트 설정도 같이 확인해야 한다.

MariaDB 재시작 명령 추가

``bind-address``를 바꾼 뒤 MariaDB 설정을 다시 적용하기 위해 아래 명령어를 사용했다.

```
sudo service mariadb restart
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 설정 파일을 수정해도 MariaDB 서비스를 재시작하지 않으면 변경 내용이 바로 적용되지 않을 수 있다. 수업에서는 ``sudo service mariadb restart`` 형태로 재시작했다.

MariaDB 사용자 확인과 계정 생성

MariaDB에 root로 접속해서 데이터베이스 목록을 확인하고, ``mysql`` 시스템 DB에서 사용자 계정을 조회했다.

```
sudo mysql -u root
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 ``sudo mysql -u root``는 리눅스 관리자 권한으로 MariaDB root 계정에 접속하는 명령이다. 접속하면 프롬프트가 ``MariaDB [(none)]>`` 형태로 바뀐다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```
show databases;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

현재 MariaDB에 만들어진 데이터베이스 목록을 확인한다. 캡처에서는 `gitea`, `information\_schema`, `mysql`, `performance\_schema`, `sys`가 보였다.

```
use mysql;
```

```
select host, user, password from user;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`mysql` 데이터베이스는 계정/권한 같은 MariaDB 내부 관리 정보가 들어있는 곳이다. `user` 테이블에서 어떤 사용자 계정이 있고 어느 host에서 접속 가능한지 확인했다.

```
create user 'dietpi'@'localhost' identified by '1234';
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`dietpi` 사용자를 만들고, `localhost`에서만 접속 가능하게 설정했다. `identified by '1234'`는 비밀번호를 지정하는 부분이다.

주의

SQL 명령은 끝에 세미콜론(`;`)이 필요하다. 처음에는 세미콜론 없이 입력해서 명령이 끝나지 않았고, 이후 다시 접속해서 정상 실행했다.

확인

`use myseqql`처럼 데이터베이스 이름을 잘못 입력하면 `ERROR 1049 (42000): Unknown database 'myseqql'`이 발생한다. 올바른 이름은 `mysql`이다.

주의

`1234`는 수업 실습용 비밀번호로만 보고, 실제 서버에서는 훨씬 긴 비밀번호와 필요한 권한만 부여하는 방식으로 설정해야 한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

MySQL Workbench에서 라즈베리파이 DB 연결

MariaDB에서 사용자/권한 관련 설정을 반영하기 위해 아래 명령을 실행했다.

```
flush privileges;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`flush privileges;`는 MariaDB/MySQL의 권한 테이블을 다시 읽어서 사용자/권한 변경사항을 반영하는 명령이다. 계정 생성, 권한 변경 후 확인용으로 자주 사용한다.

PC의 MySQL Workbench에서 새 연결을 만들었다.

```
Connection Name: RPI-DB
Username: dietpi
Hostname: 192.168.100.230
Port: 3306
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`RPI-DB`는 Workbench에서 라즈베리파이 MariaDB에 접속하기 위해 만든 연결 이름이다. 캡처 기준으로 `dietpi` 사용자로 `192.168.100.230:3306`에 연결하도록 설정했다.

흐름

라즈베리파이에서 `bind-address = 0.0.0.0`으로 외부 접속을 열고, MariaDB를 재시작한 뒤, Workbench에서 라즈베리파이 IP와 3306 포트로 접속한다.

주의

Workbench가 다른 PC에서 접속하는 경우 MariaDB 계정의 host가 `localhost`만이면 접속이 막힐 수 있다. 외부 접속이 필요하다면 수업/실습 범위에 맞게 `dietpi'@'%'` 또는 특정 PC IP 권한을 따로 확인해야 한다.

MariaDB 스키마 개념

스키마는 데이터베이스의 구조 설계도이다. 데이터를 아무렇게나 저장하는 것이 아니라, 어떤 테이블을 만들고 각 테이블에 어떤 컬럼을 둘지, 자료형과 관계를 어떻게 정할지 미리 정해 둔 틀을 말한다.

핵심

스키마는 "데이터가 들어가기 전의 구조"이다. 실제 회원 정보 한 줄이 데이터라면, 그 데이터를 어떤 모양으로 받을지 정한 틀이 스키마다.

```
CREATE TABLE user (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  email VARCHAR(100),
  created_at DATETIME
);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

위 코드는 `user`라는 테이블의 구조를 만드는 예시이다. `id`, `name`, `email`, `created\_at` 같은 컬럼이 있고, 각 컬럼마다 저장할 수 있는 데이터 종류가 정해져 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

데이터베이스	전체 저장 공간
테이블	데이터를 종류별로 나눈 표
컬럼	표의 항목 이름
행(row)	실제 데이터 한 줄
스키마	테이블, 컬럼, 자료형, 키, 관계를 정리한 설계도

흐름	데이터베이스를 만들 때는 보통 `데이터베이스 생성 -> 테이블 설계 -> 컬럼 정의 -> 기본키/외래키 설정 -> 데이터 입력` 순서로 생각한다.
확인	`show databases;`는 데이터베이스 목록 확인, `show tables;`는 테이블 목록 확인, `desc 테이블명;`은 테이블 구조 확인에 사용한다.
주의	스키마와 데이터는 다르다. 스키마는 틀이고, 데이터는 그 틀 안에 들어가는 실제 내용이다.
정리	스키마는 DB의 뼈대다. 테이블 이름, 컬럼 이름, 자료형, 키, 관계를 미리 정해서 데이터가 일정한 형식으로 저장되게 만든다.

MariaDB 외부 접속 권한 부여 보충

Workbench 같은 외부 PC 프로그램에서 라즈베리파이 MariaDB에 접속하려면 `bind-address`만 바꾸는 것으로 끝나지 않고, 외부 접속 가능한 계정 권한도 필요하다.

```
grant all privileges on *.* TO 'dietpi'@'%' identified BY '1234';
flush privileges;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석	`grant all privileges on *.* TO 'dietpi'@'%' identified BY '1234';`는 `dietpi` 계정이 외부 호스트에서도 접속할 수 있도록 권한을 주는 수업용 예시 명령이다.
주석	`%`는 모든 호스트에서의 접속을 뜻한다. 즉 Workbench를 실행하는 PC가 라즈베리파이 밖에 있어도 접속을 허용한다는 의미이다.
주석	`flush privileges;`는 MariaDB/MySQL 권한 테이블을 다시 읽어서 방금 설정한 사용자/권한 변경사항을 반영하는 명령이다.
주의	`1234`와 `*.*` 전체 권한은 수업 실습용 예시로만 본다. 실제 서버에서는 더 긴 비밀번호를 쓰고, 필요한 DB에만 권한을 주는 방식이 안전하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

2026.05.29

데이터베이스 기본 개념

데이터베이스를 엑셀에 비유하면 구조를 이해하기 쉽다. 엑셀 파일 안에 여러 시트가 있고, 각 시트에 표가 들어가는 것처럼 데이터베이스도 큰 저장 공간 안에 여러 테이블을 둔다.

DataBase 또는 Schemas

- 학생관리 >> 파일
- 학생명단 테이블 >> 시트
- 출석현황 테이블
- 기타 관리 테이블

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 MySQL Workbench에서 보이는 'SCHEMAS'는 데이터베이스 목록이라고 이해하면 된다. 하나의 스키마 안에 여러 테이블을 만들 수 있다.

핵심 데이터베이스는 데이터를 저장하는 공간이고, 테이블은 실제 데이터를 표 형태로 담는 단위이다.

DBMS

DBMS는 Database Management System의 약자로, 데이터베이스를 만들고 관리하는 프로그램이다.

MySQL	수업에서 사용한 관계형 데이터베이스 관리 시스템
Oracle	기업 환경에서 많이 쓰이는 관계형 데이터베이스 관리 시스템

정리 데이터베이스가 데이터 저장 공간이라면, DBMS는 그 저장 공간을 만들고 조회하고 수정하게 해주는 관리 프로그램이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

관계형 데이터베이스 RDB

관계형 데이터베이스는 데이터를 표 형태의 테이블로 나누고, 테이블 사이를 키로 연결해서 관리하는 방식이다.

스키마 정의	데이터베이스와 테이블 구조를 미리 설계
테이블	데이터를 저장하는 표
컬럼	저장 항목 또는 필드
레코드(row)	실제 데이터 한 줄
고유키(PK)	각 레코드를 구분하는 기본키
참고키(FK)	다른 테이블의 기본키를 참고하는 외래키

핵심 관계형 데이터베이스는 여러 표를 따로 만들고, 필요한 관계는 PK와 FK로 연결한다.

정규화

정규화는 중복되는 데이터를 줄이고, 핵심키끼리 연결되도록 테이블을 나누는 과정이다.

주석 예를 들어 학생 정보와 출석 정보를 한 표에 계속 반복해서 저장하면 같은 이름, 학번, 학과 정보가 여러 번 중복될 수 있다. 이때 학생 테이블과 출석 테이블을 나누고, 학생을 구분하는 키로 연결하면 중복을 줄일 수 있다.

정리 정규화는 데이터를 깔끔하게 나누고, 중복을 줄이고, 키를 기준으로 다시 연결하는 설계 방법이다.

SQL 명령어 분류

DDL	데이터베이스 구조를 정의하는 명령. 예: CREATE, ALTER, DROP
DML	데이터를 조회/추가/수정/삭제하는 명령. 예: SELECT, INSERT, UPDATE, DELETE
DCL	권한을 제어하는 명령. 예: GRANT, REVOKE

확인 테이블을 만드는 것은 DDL, 테이블 안의 데이터를 다루는 것은 DML, 사용자 권한을 주고 빼는 것은 DCL로 구분한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

NoSQL

NoSQL은 관계형 테이블 구조만 사용하는 방식이 아니라, 목적에 따라 더 유연한 형태로 데이터를 저장하는 방식이다.

Key-Value	key와 value 쌍으로 저장
Document	JSON 문서처럼 필드 구조를 가진 문서 단위로 저장

주의 NoSQL은 SQL을 전혀 안 쓴다는 뜻보다, 전통적인 관계형 테이블 방식만 고정해서 쓰지 않는 데이터베이스 계열로 이해하면 된다.

MySQL Workbench 스키마 생성

MySQL Workbench에서 왼쪽 `SCHEMAS` 영역을 이용해 새 스키마를 만들었다.

```
SCHEMAS 우클릭
-> Create Schema
-> Schema Name: my_community
-> Charset/Collation: utf8mb4 / utf8mb4_general_ci
-> Apply
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `my\_community`는 커뮤니티 서비스용 데이터베이스 이름으로 만든 스키마이다.

주석 `utf8mb4`는 한글과 이모지까지 저장할 수 있는 문자셋이다. 수업 메모에는 `utf8mb4 general\_`처럼 적었지만 Workbench에서는 보통 `utf8mb4`와 `utf8mb4\_general\_ci`를 선택한다.

확인 스키마 생성 후 왼쪽 `SCHEMAS` 목록에 `my\_community`가 보이면 생성된 것이다.

MySQL Workbench 테이블 생성

`my\_community` 스키마 안에서 `Tables`를 선택하고 새 테이블을 만들었다.

```
my_community
-> Tables
-> Create Table
-> Table Name: user
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

수업 캡처 기준으로 `user` 테이블에 아래 컬럼을 작성했다.

```
user_id VARCHAR(50) PRIMARY KEY NOT NULL
age INT UNSIGNED
nickname VARCHAR(200)
email VARCHAR(200)
```

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

```
password VARCHAR(300)
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석	<code>`user_id`</code> 는 사용자를 구분하는 값이므로 PK와 NN을 체크했다. PK는 Primary Key, NN은 Not Null을 의미한다.
주석	<code>`age`</code> 는 나이를 저장하는 숫자 컬럼이고, 음수가 될 필요가 없으므로 UN(Unsigned)을 체크했다.
주석	<code>`created_at`</code> 은 가입 또는 생성 시각을 저장하는 컬럼이다. 기본값을 <code>`CURRENT_TIMESTAMP`</code> 로 두면 데이터가 생성될 때 현재 시간이 자동으로 들어간다.
결과	하단 Action Output에 <code>`Changes applied`</code> 가 표시되면 Workbench에서 테이블 생성 또는 수정 적용이 완료된 것이다.
정리	오늘 실습은 <code>`my_community`</code> 스키마를 만들고, 그 안에 사용자 정보를 저장할 <code>`user`</code> 테이블 구조를 설계하는 흐름까지 진행했다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

SQL 원문과 실행 의미

수업에서 MySQL Workbench로 만든 `user` 테이블은 SQL로 보면 아래와 같은 `CREATE TABLE` 명령으로 표현된다.

```
CREATE TABLE `my_community`.`user` (
  `user_id` VARCHAR(50) NOT NULL,
  `age` INT UNSIGNED NULL,
  `nickname` VARCHAR(200) NULL,
  `email` VARCHAR(200) NULL,
  `password` VARCHAR(300) NULL,
  `created_at` DATETIME NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`user_id`)); -- 테이블생성 명령
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석	`CREATE TABLE`은 새 테이블을 만드는 DDL 명령이다. 여기서는 `my_community` 스키마 안에 `user` 테이블을 만든다.
주석	`user_id VARCHAR(50) NOT NULL`은 사용자 아이디를 최대 50글자 문자열로 저장하고, 비워둘 수 없게 설정한 컬럼이다.
주석	`age INT UNSIGNED NULL`은 나이를 정수로 저장하고, 음수는 허용하지 않으며, 값이 없어도 되는 컬럼이다.
주석	`created_at DATETIME NULL DEFAULT CURRENT_TIMESTAMP`는 생성 시간을 저장하는 컬럼이다. 값을 직접 넣지 않으면 현재 시간이 기본값으로 들어간다.
주석	`PRIMARY KEY (user_id)`는 `user_id`를 기본키로 지정한다. 기본키는 테이블에서 각 행을 구분하는 대표 값이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

데이터베이스와 테이블 확인 명령

```
show databases; -- 데이터베이스 목록 조회
use my_community; -- 사용할 데이터베이스 스키마 선택
show tables; -- 선택된 스키마의 테이블 목록 조회
select * from user; -- user 테이블의 전체 데이터 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

show databases	현재 MySQL 서버에 있는 데이터베이스/스키마 목록을 확인한다.
use my_community	앞으로 실행할 SQL 명령의 대상 스키마를 `my_community`로 선택한다.
show tables	현재 선택된 스키마 안에 있는 테이블 목록을 확인한다.
select * from user	`user` 테이블의 모든 컬럼과 모든 행을 조회한다.

확인 `select \*`에서 `\*`는 모든 컬럼을 뜻한다. 실무에서는 필요한 컬럼만 골라 조회하는 방식도 많이 사용한다.

데이터 입력

`user` 테이블에 테스트 데이터를 한 줄 추가했다.

```
insert into user (user_id, age, nickname) values ('abc', 28, '테스트');
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `insert into`는 테이블에 새 데이터를 추가하는 DML 명령이다.

주석 `(user\_id, age, nickname)`은 값을 넣을 컬럼 목록이고, `values ('abc', 28, '테스트')`는 실제로 저장할 값이다.

결과 위 명령을 실행하면 `user\_id`가 `abc`, `age`가 `28`, `nickname`이 `테스트`인 사용자 데이터가 추가된다.

주의 `user\_id`는 기본키이므로 같은 `user\_id` 값을 중복해서 넣으면 오류가 날 수 있다.

정리 오늘 SQL 흐름은 `테이블 생성 -> 스키마 선택 -> 테이블 확인 -> 데이터 조회 -> 데이터 입력` 순서로 보면 된다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

sample.sql 파일로 테이블 생성

`C:\Users\wai\Downloads\sample.sql` 파일을 MySQL Workbench에서 실행해서 커뮤니티 예제용 테이블을 만들었다.

핵심

SQL 파일은 여러 SQL 명령을 한 번에 저장해 둔 스크립트 파일이다. Workbench에서 실행하면 테이블 생성, 데이터 입력, 조회 연습 쿼리를 순서대로 실행할 수 있다.

```
-- 기존 테이블 삭제
DROP TABLE IF EXISTS comments;
DROP TABLE IF EXISTS posts;
DROP TABLE IF EXISTS users;

-- 유저 테이블
CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  nickname VARCHAR(30) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`DROP TABLE IF EXISTS`는 같은 이름의 테이블이 이미 있으면 삭제하는 명령이다. 실습을 다시 실행할 때 기존 테이블과 충돌하지 않게 한다.

주석

`comments -> posts -> users` 순서로 삭제하는 이유는 외래키 관계 때문이다. 자식 테이블을 먼저 지워야 부모 테이블 삭제 오류를 줄일 수 있다.

주석

`AUTO\_INCREMENT`는 새 행이 추가될 때 숫자 기본키가 자동으로 증가하게 하는 설정이다.

주석

`UNIQUE`는 같은 이메일이 중복 저장되지 않도록 막는 제약조건이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

게시물/댓글 테이블 생성

`posts` 테이블은 게시글을 저장하고, `comments` 테이블은 댓글을 저장한다. 두 테이블 모두 사용자 테이블의 `user\_id`를 외래키로 참고한다.

```
CREATE TABLE posts (
  post_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  title VARCHAR(200) NOT NULL,
  content TEXT NOT NULL,
  view_count INT DEFAULT 0,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
  ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE comments (
  comment_id INT AUTO_INCREMENT PRIMARY KEY,
  post_id INT NOT NULL,
  user_id INT NOT NULL,
  content VARCHAR(500) NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (post_id) REFERENCES posts(post_id),
  FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석	`FOREIGN KEY (user_id) REFERENCES users(user_id)`는 게시글/댓글 작성자가 `users` 테이블의 사용자라는 뜻이다.
주석	`ON UPDATE CURRENT_TIMESTAMP`는 행이 수정될 때 `updated_at` 시간이 자동으로 현재 시간으로 바뀌게 하는 설정이다.
정리	`users`는 부모 테이블이고, `posts`, `comments`는 `users`의 `user_id`를 참고하는 자식 테이블이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

샘플 데이터 입력

‘INSERT INTO’ 명령으로 유저, 게시물, 댓글 데이터를 여러 줄 입력했다.

```
INSERT INTO users (nickname, email) VALUES
('김민수', 'minsu@test.com'),
('이서연', 'seoyeon@test.com'),
('박지훈', 'jihoon@test.com'),
('최유진', 'yujin@test.com'),
('정하늘', 'haneul@test.com'),
('한도윤', 'doyun@test.com');
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

‘INSERT INTO users (nickname, email) VALUES ...’는 ‘users’ 테이블에 사용자 데이터를 추가하는 DML 명령이다.

확인

‘user_id’는 ‘AUTO_INCREMENT’라서 직접 넣지 않아도 1, 2, 3처럼 자동으로 생성된다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

JOIN 연습 쿼리

sample.sql에는 테이블 생성과 데이터 입력뿐 아니라 JOIN 연습용 조회 쿼리도 들어 있다.

```
SELECT
  p.post_id,
  p.title,
  u.nickname,
  p.view_count,
  p.created_at
FROM posts p
JOIN users u
  ON p.user_id = u.user_id;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`JOIN users u ON p.user\_id = u.user\_id`는 게시물의 작성자 번호와 사용자 테이블의 번호를 연결해서 게시글과 작성자 닉네임을 함께 조회하는 부분이다.

```
SELECT
  p.post_id,
  p.title,
  COUNT(c.comment_id) AS comment_count
FROM posts p
LEFT JOIN comments c
  ON p.post_id = c.post_id
GROUP BY p.post_id, p.title
ORDER BY comment_count DESC
LIMIT 3;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`LEFT JOIN`은 댓글이 없는 게시글도 결과에 남길 수 있다. `COUNT`는 댓글 개수를 세고, `GROUP BY`는 게시글별로 묶어 통계를 낸다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

Workbench에서 데이터 수정

캡처에서는 `users` 테이블 결과 그리드에서 데이터를 확인하고, 직접 수정/추가한 상태가 보였다.

```
user_id | nickname | email | password | created_at
1 | 김민수 | minsu@test.com | 1234 | 2026-05-25 11:10:57
2 | 이서연 | seoyeon@test.com | 2345 | 2026-05-26 11:10:57
3 | 박지훈 | jihoon@test.com | 3456 | 2026-05-27 11:10:57
4 | 최유진 | yujin@test.com | 4567 | 2026-05-28 11:10:57
5 | 정하늘 | haneul@test.com | 6789 | 2026-05-29 11:10:57
6 | 한도윤 | doyun@test.com | 7890 | 2026-05-29 11:10:57
7 | 김동현 | sollee8404@gmail.com | 0629 | 2026-05-29 11:18:00
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

확인

sample.sql 원본의 `users` 테이블에는 `password` 컬럼이 없으므로, 캡처의 결과는 SQL 파일 실행 후 Workbench에서 컬럼을 추가하거나 기존 테이블 구조를 수정한 상태로 볼 수 있다.

주석

Result Grid에서 직접 값을 수정한 뒤 Apply를 누르면 실제 DB에 변경사항이 반영된다.

주의

비밀번호를 실제 서비스에서 `1234`, `0629`처럼 평문으로 저장하면 위험하다. 실무에서는 비밀번호를 해시 처리해서 저장한다.

정리

오늘 흐름은 `sample.sql` 실행 -> users/posts/comments 테이블 생성 -> 샘플 데이터 입력 -> JOIN 조회 연습 -> Workbench Result Grid에서 users 데이터 수정`으로 정리할 수 있다.

MySQL SELECT, WHERE, JOIN 조회 정리

오늘은 `users` 테이블을 기준으로 SELECT 조회, WHERE 조건, LIKE 텍스트 검색, 날짜 조건, SQL 인젝션, 집계 함수, 정렬, 그룹화, JOIN까지 이어서 실습했다.

핵심

`SELECT`는 데이터를 조회하는 명령이고, `WHERE`는 조회할 행을 거르는 조건이다. 필요한 컬럼만 고르면 화면이 깔끔해지고, 조건을 붙이면 원하는 데이터만 추출할 수 있다.

기본 SELECT 조회

```
select * from users; -- users 테이블 전체조회
select nickname from users; -- users 테이블에서 닉네임 전체 조회
select user_id, nickname from users; -- users 테이블에서 아이디와 닉네임만 전체 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`\*`는 모든 컬럼을 뜻한다. `select nickname`처럼 컬럼명을 적으면 필요한 컬럼만 조회한다.

주석

`select user\_id, nickname`은 사용자 번호와 닉네임처럼 필요한 항목만 골라서 보는 방식이다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

WHERE 조건 조회

```
select * from users where user_id = 3; -- user_id가 3인 사용자 한 명만 조회
select * from users where user_id = 3 OR user_id = 5; -- user_id가 3이거나 5인 사용자 조회
select * from users where user_id in(1,2,3); -- user_id가 1, 2, 3 중 하나인 사용자 조회
select * from users where user_id <= 3; -- user_id가 3 이하인 사용자 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `where user\_id = 3`은 `user\_id` 값이 정확히 3인 행만 가져온다.

주석 `OR`은 둘 중 하나라도 조건이 맞으면 조회한다. `user\_id = 3 OR user\_id = 5`는 3번 또는 5번 사용자를 찾는다.

주석 `IN`은 여러 값 중 하나와 일치하는지 검사한다. `user\_id in(1,2,3)`은 `user\_id = 1 OR user\_id = 2 OR user\_id = 3`과 비슷한 의미이다.

주석 `<=`는 작거나 같다는 뜻이다. 숫자 범위를 조건으로 걸 때 사용한다.

텍스트 검색

```
select * from users where nickname = '이서연'; -- 닉네임이 정확히 이서연인 사용자 조회
select * from users where nickname like '이서연'; -- 닉네임이 이서연 패턴과 일치하는 사용자 조회
select * from users where nickname like '김%'; -- 김으로 시작하는 닉네임 조회
select * from users where nickname like '%수'; -- 수로 끝나는 닉네임 조회
select * from users where nickname like '%서연%'; -- 서연이 포함된 닉네임 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `=`는 값이 정확히 같은지 비교한다.

주석 `LIKE`는 문자열 패턴 검색에 사용한다. `%`가 없으면 사실상 정확히 같은 값 검색처럼 동작한다.

주석 `김%`는 김으로 시작하고 뒤에는 어떤 글자가 와도 된다는 뜻이다.

주석 `%수`는 앞에는 어떤 글자가 와도 되고 마지막이 수로 끝나는 값을 찾는다.

주석 `%서연%`은 앞뒤에 어떤 글자가 있어도 중간에 서연이 들어가면 조회한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

날짜 조건 조회

```
select * from users where created_at < '2026-05-28'; -- 2026-05-28 00:00:00 이전 데이터 추출
select * from users where created_at >= '2026-05-29' AND created_at < '2026-05-30'; -- 2026-05-29 하루 데이터만 추출
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주의 `created\_at < '2026-05-28'`은 2026-05-28 하루 전체가 아니라 `2026-05-28 00:00:00`보다 이전 데이터를 뜻한다.

핵심 특정 날짜 하루만 가져올 때는 `created\_at >= '2026-05-29' AND created\_at < '2026-05-30'`처럼 시작일은 포함하고 다음날 시작 전까지 조회하는 방식이 안전하다.

이메일 검색 예제

```
-- email주소에 test가 들어있는 사람들 추출
select * from users where email like '%test%';
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `%test%`는 이메일 앞뒤 어디든 `test` 문자열이 포함되어 있으면 조회한다.

로그인 조건 조회

```
-- 입력창: id, pw
select * from users where email = 'sollee8404@gmail.com' AND password = '0629';
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 로그인은 보통 사용자가 입력한 id/email과 password가 모두 맞는지 확인한다.

주석 `AND`는 양쪽 조건이 모두 참이어야 한다. 여기서는 이메일도 맞고 비밀번호도 맞아야 조회 결과가 나온다.

주의 실제 서비스에서는 비밀번호를 평문으로 저장하지 않는다. 보통 bcrypt 같은 방식으로 해시 처리해서 저장한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

SQL 인젝션

SQL 인젝션은 사용자가 입력한 값이 SQL 문장 일부처럼 실행되어, 의도하지 않은 로그인 우회, 데이터 조회, 테이블 삭제 등이 발생하는 공격이다.

```
-- 비밀번호 조건을 주석 처리해서 강제 로그인처럼 만드는 예
select * from users where email = 'sollee8404@gmail.com'; -- ; AND password = '0629';

-- 뒤에 DROP TABLE을 붙여 테이블 삭제를 시도하는 예
select * from users where email = 'sollee8404@gmail.com'; DROP TABLE users; -- AND password = '0629';

-- 항상 참인 조건을 붙여 전체 사용자가 조회될 수 있는 예
select * from users where email = 'sollee8404@gmail.com' OR 1=1; -- ; AND password = '0629';
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주의

'OR 1=1'은 항상 참이다. 로그인 SQL에 이런 입력이 섞이면 비밀번호가 틀려도 조건 전체가 참이 되어버릴 수 있다.

주의

'--'는 SQL에서 주석으로 쓰인다. 공격자가 뒤쪽 조건을 주석 처리하면 원래 확인해야 할 비밀번호 조건이 무시될 수 있다.

SQL 인젝션 방어 방법

핵심

사용자 입력값을 SQL 문자열에 직접 이어붙이지 말고, 파라미터 바인딩 또는 ORM을 사용한다.

파라미터 바인딩	SQL 문장과 입력값을 분리해서 DB에 전달하는 방식
Prepared Statement	미리 SQL 구조를 정해두고 값만 나중에 안전하게 넣는 방식
ORM	객체와 테이블을 연결해서 SQL을 직접 조립하는 실수를 줄여주는 도구

주석

ORM은 Object Relational Mapping의 약자이다. Java의 JPA, Node.js의 Sequelize/Prisma, Python의 SQLAlchemy 같은 도구가 ORM에 해당한다.

정리

실무에서는 사용자가 입력한 이메일, 비밀번호, 검색어를 SQL 문자열에 직접 붙이지 않고 ORM이나 파라미터 바인딩으로 처리한다고 이해하면 된다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

테이블 데이터 개수 구하기

```
select count(*) from users;
select count(*) from users where email like '%test%';
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `count(\*)`는 조건에 맞는 행의 개수를 센다.

주석 첫 번째 쿼리는 전체 사용자 수를 구하고, 두 번째 쿼리는 이메일에 `test`가 들어간 사용자 수만 구한다.

정렬해서 가져오기

```
select * from users order by nickname;
select * from users where email like '%test%' order by nickname desc; -- 기본정렬 asc, 반대정렬 desc
select * from users where email like '%test%' order by nickname asc, email desc;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `order by nickname`은 닉네임 기준으로 정렬한다. 기본값은 오름차순 `ASC`이다.

주석 `DESC`는 내림차순 정렬이다.

주석 `order by nickname asc, email desc`는 먼저 닉네임을 오름차순으로 정렬하고, 닉네임이 같으면 이메일을 내림차순으로 정렬한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

중복 제거와 그룹화

```
select distinct(nickname) from users; -- 중복을 제거한 닉네임 목록 조회
select nickname, count(nickname) from users group by nickname; -- 닉네임별 데이터 개수 조회
select nickname, count(nickname), max(user_id) from users group by nickname; -- 닉네임별 개수와 가장 큰 user_id 조회
select nickname, count(nickname), max(user_id), min(user_id) from users group by nickname; -- 닉네임별 개수, 최대 user_id, 최소 user_id 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `distinct(nickname)`은 같은 닉네임이 여러 번 있어도 한 번만 보여준다.

주석 `group by nickname`은 같은 닉네임끼리 묶는다. 묶은 뒤 `count(nickname)`으로 닉네임별 개수를 셀 수 있다.

주석 `max(user\_id)`는 그룹 안에서 가장 큰 `user\_id`를 구한다.

주석 `min(user\_id)`는 그룹 안에서 가장 작은 `user\_id`를 구한다.

핵심 `GROUP BY`를 쓰면 행을 하나씩 보는 것이 아니라 같은 기준끼리 묶어서 통계를 낼 수 있다.

게시물 전체 조회

```
select * from posts;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `posts` 테이블의 모든 게시물 데이터를 조회한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

게시물과 작성자 같이 조회하기: Sub Query

```
select
post_id,
(select nickname from users where users.user_id = p.user_id) AS user_name,
title, view_count, created_at
from posts AS p;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 서브쿼리는 SQL 안에 들어가는 또 다른 SELECT 문이다.

주석 `(select nickname from users where users.user\_id = p.user\_id)`는 게시물 한 줄마다 해당 작성자의 닉네임을 users 테이블에서 찾아온다.

주석 `AS user\_name`은 조회 결과 컬럼 이름을 `user\_name`으로 보여주기 위한 별칭이다.

주석 `from posts AS p`는 `posts` 테이블에 `p`라는 짧은 별칭을 붙인 것이다.

게시물과 작성자 같이 조회하기: JOIN

```
select p.post_id, u.nickname, p.title, p.view_count
from posts AS p
join users AS u ON p.user_id = u.user_id;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `JOIN`은 두 테이블을 동시에 조회하면서, `ON` 조건이 일치하는 데이터끼리 묶어 보여준다.

주석 `p.user\_id = u.user\_id`는 게시물 테이블의 작성자 번호와 사용자 테이블의 사용자 번호가 같은 행끼리 연결한다는 뜻이다.

핵심 작성자 닉네임처럼 다른 테이블에 있는 정보를 함께 보고 싶을 때 JOIN을 사용한다. 서브쿼리로도 가능하지만, 테이블 관계를 묶어서 조회할 때는 JOIN이 더 자주 쓰인다.

정리 오늘 SQL 조회 흐름은 `SELECT 기본 조회 -> WHERE 조건 -> LIKE 검색 -> 날짜 범위 -> 로그인 조건 -> SQL 인젝션 주의 -> COUNT/ORDER BY/GROUP BY -> 서브쿼리와 JOIN` 순서로 보면 된다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

2026.06.04

데이터베이스 조회와 JOIN

핵심

오늘 내용은 `users`, `posts` 테이블을 기준으로 데이터 조회, 실행 계획 확인, subquery와 JOIN의 차이, LEFT/RIGHT JOIN으로 누락 데이터를 찾는 흐름이다.

확인

초반의 `show databases`, `use`, `show tables`, 기본 `select`는 기존 데이터베이스 복습 내용이므로 짧게 압축하고, 새로 중요한 부분인 `LIMIT`, `explain`, subquery 성능, JOIN 결과 차이를 중심으로 정리한다.

복습: 데이터베이스 선택과 기본 조회

```
show databases; -- 데이터베이스 목록 조회
use my_community; -- 사용할 기본 데이터베이스 선택

-- 테이블 목록 조회
show tables;

-- users 테이블 데이터 수 조회
select count(*) from users;

-- users 테이블 내용 조회
select * from users; -- users 테이블 조회 >> 너무 많으면 위험할수도있음
select * from users LIMIT 5; -- 5개 제한 조회
select * from users order by user_id desc LIMIT 5; -- user_id 역순 5개 조회
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`select \* from users`는 모든 행과 모든 열을 가져오므로 데이터가 많으면 위험할 수 있다. 실습이나 확인용이면 `LIMIT`을 붙여서 일부만 조회하는 습관이 좋다.

주석

`order by user\_id desc LIMIT 5`는 `user\_id`가 큰 순서, 즉 최근에 만들어진 사용자부터 5개를 보는 형태로 사용할 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

조건 조회와 인덱스 주의

```
select * from users where email = 'sollee8404@gmail.com' and password = '0629';
select * from users where password = '0629' and email = 'sollee8404@gmail.com'; -- 폴스캔, 이렇게 짜면 안됨 email은 unique라
인덱스값으로 찾는데 password는 인덱스가 없어서 전체를 찾음
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `email`이 `unique`라면 DB가 인덱스를 이용해 빠르게 사용자를 찾을 수 있다. 반대로 `password`에는 보통 인덱스가 없기 때문에 password 조건을 중심으로 찾으면 전체 데이터를 훑는 폴스캔 위험이 있다.

주의 실제 서비스에서는 비밀번호를 평문으로 조회/저장하면 안 된다. 수업 SQL은 조건 조회와 인덱스 설명용 예시로 보고, 운영에서는 해시된 비밀번호를 비교해야 한다.

실행 계획 확인

```
-- 실행 계획 확인
explain select * from users;
explain select * from users where user_id = 3;
explain select * from users where user_id > 5;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `explain`은 DB가 SQL을 어떻게 실행할지 보여준다. 인덱스를 타는지, 전체 테이블을 훑는지 확인할 때 사용한다.

주석 `user\_id = 3`처럼 기본키/인덱스 조건이면 특정 행을 빠르게 찾을 수 있고, `user\_id > 5`는 범위 조건으로 여러 행을 탐색한다.

게시물 목록 조회

```
-- 게시물 목록을 출력 (작성자(닉네임), 제목, 조회수, 등록일자)
select * from posts;
select user_id, title, view_count, created_at from posts;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 게시물 목록 화면에서는 모든 컬럼이 아니라 필요한 컬럼만 가져오는 것이 좋다. 예시는 작성자 식별값, 제목, 조회수, 등록일자를 조회한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

Subquery로 작성자 닉네임 가져오기

```
-- subquery
-- 장점: 단순하다
-- 단점: posts 데이터 수만큼 서브쿼리가 실행됨 >> 성능하락

select
(select nickname from users u where u.user_id = p.user_id) AS nink,
title,
view_count,
created_at
from
posts AS p; -- 테이블 뒤에 AS생략가능 users u >> users AS u
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 subquery는 SQL 구조가 단순해서 이해하기 쉽다. 하지만 게시물마다 내부 쿼리가 반복 실행될 수 있어 데이터가 많아지면 성능이 떨어질 수 있다.

주석 `posts AS p`, `users u`처럼 테이블에 별칭을 붙이면 SQL을 짧게 쓸 수 있다. `AS`는 생략할 수 있다.

JOIN으로 users와 posts 연결

```
-- JOIN (inner join = 교집합) -- on뒤에는 교집합 조건
select * from posts JOIN users on posts.user_id = users.user_id;
select * from posts INNER JOIN users on posts.user_id = users.user_id;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `INNER JOIN`은 양쪽 테이블에 모두 매칭되는 데이터만 가져온다. 즉 게시물의 `user\_id`와 사용자 테이블의 `user\_id`가 일치하는 행만 결과에 나온다.

주석 `JOIN`만 써도 보통 `INNER JOIN`과 같은 의미로 동작한다.

LEFT JOIN / RIGHT JOIN

```
-- 탈퇴하고 없는 사람 글도 조회
select * from posts LEFT JOIN users on posts.user_id = users.user_id; -- 설명추가
select * from posts RIGHT JOIN users on posts.user_id = users.user_id; -- 설명추가
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `LEFT JOIN`은 왼쪽 테이블인 `posts`를 기준으로 전부 보여주고, 매칭되는 사용자가 있으면 붙인다. 사용자가 삭제되어도 글을 기준으로 확인할 수 있다.

주석 `RIGHT JOIN`은 오른쪽 테이블인 `users`를 기준으로 전부 보여주고, 매칭되는 게시물이 있으면 붙인다. 게시물을 작성하지 않은 사용자도 확인할 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

탈퇴한 사람 글만 찾기

```
-- 탈퇴한 사람 글만 조회
select * from posts LEFT JOIN users on posts.user_id = users.user_id
WHERE users.user_id is null;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `posts`에는 글이 남아 있는데 `users`에서 매칭되는 사용자가 없으면 `users.user\_id`가 `null`이 된다. 이 조건으로 탈퇴한 사용자의 글만 찾을 수 있다.

게시물 작성 내역이 없는 사람 찾기

```
-- 게시물 작성 내역이 없는사람
select * from posts RIGHT JOIN users on posts.user_id = users.user_id
WHERE posts.post_id is null;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `users`에는 사용자가 있는데 `posts`에서 매칭되는 게시물이 없으면 `posts.post\_id`가 `null`이 된다. 이 조건으로 글을 작성하지 않은 사용자를 찾을 수 있다.

JOIN 그림 요약

INNER JOIN	A와 B의 교집합만 조회
LEFT JOIN	왼쪽 A는 전부 조회하고, B는 매칭되는 것만 붙임
RIGHT JOIN	오른쪽 B는 전부 조회하고, A는 매칭되는 것만 붙임

- LEFT JOIN + `WHERE B.key IS NULL`: A에는 있지만 B에는 없는 데이터
- RIGHT JOIN + `WHERE A.key IS NULL`: B에는 있지만 A에는 없는 데이터

FULL OUTER JOIN	A와 B 전체를 합쳐서 조회한다. MySQL에서는 기본 문법으로 직접 지원하지 않아 `LEFT JOIN`과 `RIGHT JOIN`을 `UNION`으로 조합하는 방식이 필요할 수 있다.
------------------------	--

정리 게시글과 사용자처럼 서로 연결된 테이블을 다룰 때는 subquery보다 JOIN을 우선 고려한다. JOIN은 어떤 테이블을 기준으로 삼는지, 매칭되지 않는 데이터를 남길지 버릴지를 기준으로 선택한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

데이터베이스 복습: CRUD와 게시판 구조

핵심

데이터베이스를 배우는 이유는 게시판 같은 서비스를 만들기 위해서다. 게시판은 글/댓글 데이터를 저장하고, 사용자를 인증하고, 서버와 통신하며, 필요하면 실시간 기능까지 붙는다.

CRUD	생성(Create), 조회(Read), 갱신(Update), 삭제>Delete)
게시판	데이터 CRUD + 인증 + 통신(REST API) + 웹소켓
웹소켓	채팅처럼 실시간으로 주고받는 기능에 사용

주석

게시판을 만들 때 DB는 글, 댓글, 사용자, 조회수, 작성일 같은 데이터를 저장하고 꺼내는 핵심 역할을 한다.

조회: SELECT

```
SELECT 컬럼명1, 컬럼명2
FROM 테이블명
WHERE 필터링조건;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

수업 메모의 `SELCET`는 `SELECT` 오타로 정리했다. `SELECT`는 원하는 컬럼을 조회하고, `WHERE`는 필요한 행만 필터링한다.

주석

게시물과 댓글을 한 번에 보여주려면 서로 다른 테이블을 연결해야 하므로 `JOIN`을 사용한다.

생성, 갱신, 삭제

```
-- 생성(등록)
INSERT INTO 테이블명 (컬럼1, 컬럼2)
VALUES (값1, 값2);

-- 갱신(업데이트)
UPDATE 테이블명
SET 컬럼명 = 새값
WHERE 조건;

-- 삭제
DELETE FROM 테이블명
WHERE 조건;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주의

`UPDATE`와 `DELETE`는 `WHERE` 없이 실행하면 테이블 전체가 바뀌거나 삭제될 수 있다. 실습할 때도 먼저 `SELECT ... WHERE ...`로 대상이 맞는지 확인하는 습관이 좋다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

데이터베이스 Index 개념

인덱스는 책의 색인처럼 특정 컬럼 값을 빠르게 찾기 위한 별도 자료구조다. 예를 들어 `users.email`에 인덱스가 있으면 DB가 모든 사용자를 처음부터 끝까지 훑지 않고, 인덱스를 통해 해당 이메일 위치를 빠르게 찾을 수 있다.

장점	`WHERE`, `JOIN`, `ORDER BY`에서 검색 속도가 빨라질 수 있음
단점	인덱스도 저장 공간을 차지함
단점	`INSERT`, `UPDATE`, `DELETE` 때 인덱스도 같이 수정해야 하므로 쓰기 성능이 느려질 수 있음
기준	자주 검색하거나 JOIN 조건에 쓰는 컬럼에 인덱스를 만든다
주의	값 종류가 너무 적은 컬럼은 인덱스 효율이 낮을 수 있다

-- 예시: email로 자주 사용자를 찾는 경우

```
CREATE INDEX idx_users_email ON users(email);
```

-- 예시: posts와 users를 user_id로 자주 JOIN하는 경우

```
CREATE INDEX idx_posts_user_id ON posts(user_id);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`email`처럼 유일하거나 선택도가 높은 컬럼은 인덱스 효율이 좋다. 반대로 성별처럼 값 종류가 적은 컬럼은 인덱스를 만들어도 효과가 작을 수 있다.

인덱스를 두 개 만들면?

인덱스를 두 개 만든다는 것은 DB가 검색할 때 사용할 수 있는 길을 두 개 만들어 두는 것이다. 예를 들어 `email` 인덱스와 `created\_at` 인덱스가 있으면, 이메일 검색에는 `email` 인덱스를 쓰고 최신순 정렬/범위 조회에는 `created\_at` 인덱스를 쓸 수 있다.

```
CREATE INDEX idx_users_email ON users(email);
```

```
CREATE INDEX idx_users_created_at ON users(created_at);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

인덱스가 여러 개 있다고 해서 한 쿼리에서 항상 전부 쓰는 것은 아니다. DB 옵티마이저가 실행 계획을 보고 가장 유리한 인덱스를 고른다.

주의

인덱스를 많이 만들수록 조회 선택지는 늘지만, 데이터 등록/수정/삭제 때 관리해야 할 인덱스도 늘어난다. 그래서 "많을수록 좋다"가 아니라 "자주 쓰는 조회 조건에 맞게 필요한 만큼" 만드는 것이 중요하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

복합 인덱스

두 컬럼을 따로 인덱스로 만드는 것과, 두 컬럼을 묶어서 하나의 복합 인덱스로 만드는 것은 다르다.

```
-- 각각 따로 만드는 인덱스
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_password ON users(password);

-- 두 컬럼을 묶은 복합 인덱스
CREATE INDEX idx_users_email_password ON users(email, password);
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `(email, password)` 복합 인덱스는 `email` 조건부터 사용하는 쿼리에 유리하다. 인덱스는 앞 컬럼부터 정렬된 구조라서 컬럼 순서가 중요하다.

주의 실제 서비스에서 `password`를 인덱싱하거나 평문으로 비교하는 방식은 좋지 않다. 수업에서는 인덱스 동작을 이해하기 위한 예시로 보고, 운영에서는 비밀번호 해시와 인증 로직을 사용한다.

MySQL Workbench 조회 제한 설정

```
Edit > Preferences... > SQL Editor > SQL Execution > Limit Rows Count
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 Workbench에서 조회 결과 행 수 제한을 설정할 수 있다. 실수로 `SELECT \* FROM users;`처럼 많은 데이터를 조회했을 때 너무 많은 행을 가져오는 것을 줄이는 안전장치다.

정리 게시판 DB를 다룰 때는 CRUD 문법을 기본으로 알고, 조회가 많아지는 컬럼에는 인덱스를 고려한다. 다만 인덱스는 조회를 빠르게 하는 대신 쓰기 비용과 저장 공간을 늘리므로, `EXPLAIN`으로 실제 실행 계획을 확인하면서 설계해야 한다.

데이터 등록, 수정, 삭제와 댓글 조회 문제

핵심 앞부분의 `show databases`, `users/posts` 조회, subquery/JOIN 복습은 기존 260604 정리에 이미 들어갔다. 이번 추가 정리에서는 새로 나온 `INSERT`, `UPDATE`, `DELETE`, `SAFE UPDATE`, `comments` 테이블 문제 풀이를 중심으로 정리한다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

데이터 등록: INSERT

```
-- 데이터 등록
INSERT INTO users (nickname, email, password)
values ('등록유저', 'aaa@naver.com', '1234');
select * from users;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `INSERT INTO users (nickname, email, password)`는 `users` 테이블의 지정한 컬럼에 새 행을 추가한다. 등록 후 `select \* from users;`로 추가된 사용자를 확인했다.

주의 `select \*`는 데이터가 많으면 위험할 수 있으므로 확인용이면 `LIMIT`을 붙이는 것이 좋다.

데이터 수정: UPDATE

```
-- 업데이트 >> WHERE 구분없이 UPDATE를 막는 설정방법도 알려줘(SAFE UPDATE) // 잘못 수정을 대비해서 무조건 SELECT로 먼저
SELECT * FROM users WHERE user_id = 8;
UPDATE users SET nickname = '홍길동' WHERE user_id = 8;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 수정하기 전에는 반드시 같은 `WHERE` 조건으로 `SELECT`를 먼저 실행해서 대상 행이 맞는지 확인한다. 그 다음 `UPDATE`를 실행한다.

주의 `UPDATE users SET nickname = '홍길동';`처럼 `WHERE` 없이 실행하면 모든 사용자의 닉네임이 바뀔 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

SAFE UPDATE 설정

SAFE UPDATE는 실수로 조건 없는 `UPDATE`나 `DELETE`를 실행하는 일을 막기 위한 안전 설정이다.

```
-- 현재 세션에서 safe update 켜기
SET SQL_SAFE_UPDATES = 1;

-- 현재 설정 확인
SELECT @@SQL_SAFE_UPDATES;

-- 필요할 때만 끄기
SET SQL_SAFE_UPDATES = 0;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `SQL\_SAFE\_UPDATES = 1`이면 key 컬럼 조건이 없거나 제한 조건이 부족한 `UPDATE`, `DELETE`를 막아준다. 실습 중 전체 수정/삭제 사고를 줄이는 안전장치다.

MySQL Workbench 설정 경로

Edit > Preferences... > SQL Editor > Safe Updates

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 Workbench에서 Safe Updates 옵션을 켜면 GUI 환경에서도 위험한 수정/삭제를 막는 데 도움이 된다. 설정을 바꾼 뒤에는 연결을 다시 열어야 적용되는 경우가 있다.

데이터 삭제: DELETE

```
-- 삭제 >> 잘못 삭제를 대비해서 SELECT로 먼저
SELECT * FROM users WHERE user_id = 8;
DELETE FROM users WHERE user_id = 8;
-- 주의!! 인생 망함 >> DELETE FROM users;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 삭제도 수정과 마찬가지로 먼저 `SELECT`로 삭제 대상이 정확한지 확인한 다음 `DELETE`를 실행한다.

주의 `DELETE FROM users;`는 `users` 테이블의 모든 행을 삭제한다. 조건 없는 DELETE는 매우 위험하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

문제 1: comments 테이블 조회

```
-- 문제
-- 1. comments 테이블 데이터를 모두 조회하시오
SELECT * FROM comments;

-- 1.1 3번 게시글의 댓글만 조회
SELECT * FROM comments WHERE post_id = 3;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `comments` 전체를 확인한 뒤, `WHERE post\_id = 3`으로 3번 게시글에 달린 댓글만 필터링한다.

문제 2-1: 댓글 내용과 작성 시각 조회

```
-- 2. comments 테이블에서 다음 항목을 조회하시오
-- 2-1 : content, create_at
SELECT content, created_at FROM comments;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 필요한 컬럼만 지정해서 조회한다. 메모에는 `create\_at`이라고 적었지만 실제 SQL은 `created\_at` 컬럼을 사용했다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

문제 2-2: 게시물 제목과 댓글 내용 조회

-- 2-2 : 댓글목록조회: 시물제목(posts.title), content, create_at(comments) // 서브쿼리, 조인

```
SELECT * FROM posts;
```

-- SUBQUERY

```
SELECT
(SELECT title from posts AS p WHERE p.post_id = c.post_id) AS title,
content,
created_at
FROM
comments AS c
WHERE
post_id = 3;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

subquery 방식은 `comments`의 각 행마다 `posts`에서 제목을 찾아 붙인다. 구조는 단순하지만 댓글 수가 많아지면 반복 조회 때문에 성능이 떨어질 수 있다.

-- JOIN

```
SELECT
p.title,
c.content,
c.created_at
FROM comments AS c
JOIN posts AS p ON p.post_id = c.post_id
WHERE
c.post_id = 3;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

JOIN 방식은 `comments.post\_id`와 `posts.post\_id`를 기준으로 두 테이블을 연결한다. 게시물 제목과 댓글 내용을 함께 조회할 때 일반적으로 더 적절하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

문제 2-3: 게시물 제목, 댓글 작성자, 댓글 내용, 작성 시각

```
-- 2-3 : 댓글목록조회: 게시물제목, 댓글작성자닉네임, 댓글내용, 댓글작성시각
-- SUBQUERY
SELECT
(SELECT title from posts AS p WHERE p.post_id = c.post_id) AS title,
(SELECT nickname from users AS u WHERE u.user_id = c.user_id) AS nickname,
content,
created_at
FROM
comments AS c
WHERE
post_id = 3;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

subquery를 두 번 사용해서 게시물 제목과 사용자 닉네임을 각각 가져온다. 이해는 쉽지만 댓글 행마다 `posts`, `users` 조회가 반복될 수 있다.

```
-- JOIN
SELECT
p.title,
u.nickname,
c.content,
c.created_at
FROM comments c
JOIN posts p ON c.post_id = p.post_id
JOIN users u ON c.user_id = u.user_id
WHERE c.post_id = 3;
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

`comments`, `posts`, `users` 세 테이블을 JOIN해서 댓글 목록 화면에 필요한 정보를 한 번에 가져온다. 게시판 댓글 목록에서는 이 방식이 실제 화면 구성에 가깝다.

정리

CRUD는 게시판의 기본 동작이고, JOIN은 화면에 필요한 여러 테이블의 데이터를 합쳐 보여주는 핵심 문법이다. 수정/삭제는 반드시 먼저 `SELECT`로 대상 확인 후 실행하고, 가능하면 SAFE UPDATE를 켜서 실수를 줄인다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

2026.06.05

Express 라우트 순서 문제

핵심

Express는 위에서 아래로 라우트를 검사한다. 그래서 구체적인 경로(`/posts/form`)보다 동적 경로(`/posts/:postId`)가 먼저 있으면 `/posts/form` 요청도 `/posts/:postId`에 걸린다.

문제 원인

```
app.get('/posts/:postId', ...)
app.get('/posts/form', ...)
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석

이 순서에서는 `/posts/form`으로 접속해도 Express가 먼저 `/posts/:postId`와 매칭한다. 그 결과 `postId` = "form"이 되고, DB에서 `post\_id`가 "form"인 글을 찾으려고 해서 게시글을 못 찾는다.

주의

라우트에서 `postId` 같은 파라미터는 거의 모든 문자열을 받을 수 있으므로, 고정 경로보다 아래에 두는 것이 안전하다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

수정 코드: 고정 경로를 먼저 선언

```
app.get('/posts/form', function(req, res){
  res.render('form');
});

app.get('/posts/:postId', function(req, res){
  const postId = req.params.postId;

  db.query("SELECT * FROM posts WHERE post_id = ?", [postId], function(err, posts){
    if (err) {
      console.log(err);
      return res.send('DB 오류');
    }

    if (posts.length === 0) {
      return res.send('게시글 없음');
    }

    res.render('post', { post: posts[0] });
  });
});
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 ``/posts/form``처럼 정확히 정해진 주소를 먼저 등록하고, ``/posts/:postId``처럼 값이 변하는 동적 주소를 나중에 등록한다.

주석 ``req.params.postId``는 URL의 ``:postId`` 부분에 들어온 값을 가져온다. ``/posts/3``이면 ``postId``는 ``"3"``이고, 잘못된 순서에서는 ``/posts/form``도 ``postId``가 ``"form"``이 된다.

주석 SQL의 ``?``와 ``[postId]``는 prepared statement 방식이다. 직접 문자열을 이어 붙이지 않고 값을 따로 넘겨 SQL injection 위험을 줄인다.

주석 ``posts.length === 0``이면 조회 결과가 없는 것이므로 ``게시글 없음``을 반환한다. 결과가 있으면 첫 번째 행인 ``posts[0]``을 ``post.ejs``에 넘긴다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

post.ejs 출력 코드

```
<h5><%= post.title %></h5>
<pre class="content"><%= post.content %></pre>
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `<%= post.title %>`은 서버에서 넘긴 `post` 객체의 제목을 출력한다. `<%= post.content %>`는 본문을 출력한다.

주석 `<pre>` 태그는 줄바꿈과 공백을 비교적 그대로 보여주기 때문에 게시글 본문 출력에 사용할 수 있다.

정리 Express 라우트는 작성 순서가 중요하다. `posts/form` 같은 고정 라우트는 `posts/:postId` 같은 동적 라우트보다 위에 뒤야 한다.

라우트 순서 개념: 넓은 조건이 먼저 잡아먹는 문제

핵심 이 문제는 단순히 "라우터 문제"라기보다, 넓은 조건을 먼저 쓰면 구체적인 조건까지 먼저 잡아먹는 문제다. Express 라우트도 `if / else if`처럼 위에서 아래로 먼저 맞는 조건을 찾는다.

C 언어 if문으로 비유

```
int score = 95;

if (score >= 0) {
    printf("점수 있음");
} else if (score >= 90) {
    printf("A");
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `95`는 `score >= 90`에도 맞지만, 위에 있는 더 넓은 조건 `score >= 0`에 먼저 걸린다. 그래서 `A`까지 가지 못한다.

```
int score = 95;

if (score >= 90) {
    printf("A");
} else if (score >= 0) {
    printf("점수 있음");
}
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 더 구체적인 조건인 `score >= 90`을 먼저 검사해야 원하는 결과가 나온다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

Python 조건문으로 비유

```
path = "/posts/form"

if path.startswith("/posts/"):
    print("게시글 상세")
elif path == "/posts/form":
    print("글쓰기 폼")
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 ``/posts/form``은 ``path.startswith("/posts/")``에도 맞기 때문에 아래의 ``path == "/posts/form"``까지 가지 못한다.

```
path = "/posts/form"

if path == "/posts/form":
    print("글쓰기 폼")
elif path.startswith("/posts/"):
    print("게시글 상세")
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 고정된 정확한 조건을 먼저 두고, 넓은 조건을 나중에 뒤야 한다.

Express에서 실제로 생긴 문제

```
app.get('/posts/:postId', ...)
app.get('/posts/form', ...)
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 ``/posts/:postId``는 ``/posts/`` 뒤에 오는 값을 전부 ``postId``로 받을 수 있는 넓은 조건이다. 그래서 ``/posts/form``으로 들어가도 먼저 ``/posts/:postId``에 걸린다.

```
req.params.postId = "form";
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 Express가 ``postId``를 모르는 것이 아니라, ``postId``를 ``"form"``이라고 잘못 알아듣는 것이 문제다.

```
SELECT * FROM posts WHERE post_id = 'form';
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 ``post_id``는 보통 숫자인데 ``"form"``으로 조회하니 게시글을 찾지 못한다. 그 결과 ``posts[0]``이 ``undefined``가 되고, ``post.ejs``에서 ``post.title``을 출력할 때 문제가 생길 수 있다.

subject	title	check ☐ ☐ ☐ ☐ ☐
김태형교수님	김태형교수님 수업정리	date

올바른 라우트 순서

```
app.get('/posts/form', function(req, res){
  res.render('form');
});

app.get('/posts/:postId', function(req, res){
  const postId = req.params.postId;
  // 게시물 상세 조회
});
```

위 명령어/코드는 수업 중 사용한 실행 흐름을 그대로 따라가기 위한 예시.

주석 `/posts/form`처럼 정확히 정해진 고정 라우트를 먼저 두고, `/posts/:postId`처럼 넓게 받는 동적 라우트를 나중에 둔다.

정리 `if / else if`, Python 조건문, Express 라우트는 모두 위에서 아래로 먼저 맞는 조건을 처리한다. 그래서 좁고 구체적인 조건을 먼저, 넓고 일반적인 조건을 나중에 뒤야 한다.